PROJECT REPORT – CUSTOM LS

Name: Riyaan Sharma
Admission Number: 23JE0819
Group Number: 25
Course: Operating Systems Lab
Project: Custom UNIX-like Utilities in C

Introduction

This project focuses on implementing simplified versions of UNIX utilities using the C programming language, along with a custom-built shell that executes these utilities. The goal is to understand the internal working of commonly used UNIX commands by building them from scratch.

As part of this project, I developed **custom_ls**, a lightweight implementation of the UNIX ls command. The ls command is one of the most frequently used utilities in UNIX systems and is responsible for listing directory contents.

The custom implementation replicates core functionalities such as displaying file names, showing hidden files, and presenting detailed file information. Additionally, enhanced features such as sorting, color-coded output, and human-readable file sizes have been implemented to improve usability and closely mimic real-world UNIX behavior.

Objective

The main objectives of this implementation are:

- To develop a custom version of the UNIX ls command
- To understand directory traversal using system calls
- To retrieve and interpret file metadata using system-level APIs
- To implement command-line flags and argument parsing
- To design modular and maintainable C programs
- To enhance usability with additional features such as sorting and color output

---

Features of custom_ls

Core Features

- Lists files and directories in the current or specified path
- Supports multiple directory arguments
- Displays output similar to UNIX ls

Flag Support

- -l → Long listing format (permissions, size, time)
- -a → Includes hidden files (starting with .)
- -h → Displays file sizes in human-readable format

Advanced Features

- Supports **combined flags**: -lh, -la, -lah
- Files are **sorted alphabetically**
- **Color-coded output**:
    - Blue → Directories
    - Green → Executable files
- Displays:
    - File permissions
    - Number of links
    - File size
    - Last modified time

Summary Output

Files: X | Directories: Y

Error Handling

- Gracefully handles invalid directory paths
- Displays meaningful error messages

Concepts Used

This project uses several important system-level programming concepts:

Directory Handling

- opendir() → Opens directory stream
- readdir() → Reads directory entries
- closedir() → Closes directory

File Metadata

- stat() → Retrieves file information such as:
    - Size
    - Permissions
    - Last modified time

Bitwise Operations

- Used to extract permission bits (rwx) from mode_t

Command-Line Parsing

- argc, argv used to process flags and arguments

Sorting

- qsort() used to sort file names alphabetically

Time Formatting

- strftime() used to display readable timestamps

ANSI Escape Codes

- Used for colored terminal output

Code Structure & Explanation

The code is modular and divided into multiple functions:

print_permissions(mode_t mode)

This function extracts permission bits from the file mode and prints them in the format:

-rwxr-xr-x

It uses bitwise operations to check read, write, and execute permissions for:

- Owner

- Group
- Others

print_colored_name(const char *name, mode_t mode)

This function prints file names with color coding:

- Directories → Blue
- Executable files → Green
- Normal files → Default

This improves readability and mimics real UNIX ls.

print_size(long size)

Displays file size in human-readable format:

Size	Output
<1024	Bytes
<1MB	KB
≥1MB	MB

compare()

Used by qsort() to sort directory entries alphabetically using strcmp().

list_directory()

This is the **core function**:

Steps performed:

1. Opens directory using opendir()
2. Reads entries using readdir()
3. Filters hidden files (if -a not set)
4. Stores entries in an array
5. Sorts entries using qsort()
6. Retrieves metadata using stat()
7. Displays output based on flags
8. Counts files and directories
9. Prints summary

parse_flags()

Handles command-line arguments and supports combined flags like:

-lh, -la, -lah

Each character after - is processed individually.

main()

- Parses flags
- Determines directory path
- Calls list_directory()
- Acts as the entry point

Execution Steps

🔹 **Compile**

```
make
```

🔹 **Run Directly**

```
./custom_ls
```

🔹 **Run Using Shell**

```
./custom_shell
custom_ls
```

Usage Examples

```
custom_ls
custom_ls -a
custom_ls -l
custom_ls -lh
custom_ls -lah
custom_ls ..
```

Screenshots

Screenshot 1 – Basic Listing

Command: **_custom_ls**

```
riyaan-sharma@riyaan-sharma:~/G25_OS-Project/G25_Project2_1$ ./custom_shell
Welcome to Custom Shell! Type 'help' for commands.
custom_shell:~/G25_OS-Project/G25_Project2_1$ custom_ls
Adyant.pdf
Makefile
Nayan.pdf
README.md
custom_cat
custom_cat.c
custom_grep
custom_grep.c
custom_ls
custom_ls.c
custom_shell
custom_shell.c
sahil.pdf

Files: 13 | Directories: 0
custom_shell:~/G25_OS-Project/G25_Project2_1$
```

Screenshot 2 – Long Listing

Command: **custom_ls -l**

```
custom_shell:~/G25_OS-Project/G25_Project2_1$ custom_ls -l
-rw-rw-r-- 1 4067917 Apr 12 16:42 Adyant.pdf
-rw-rw-r-- 1  2313 Apr 12 16:42 Makefile
-rw-rw-r-- 1 543901 Apr 12 16:42 Nayan.pdf
-rw-rw-r-- 1 10405 Apr 12 16:42 README.md
-rwxrwxr-x 1 16528 Apr 12 16:56 custom_cat
-rw-rw-r-- 1  3399 Apr 12 16:42 custom_cat.c
-rwxrwxr-x 1 16496 Apr 12 16:56 custom_grep
-rw-rw-r-- 1  1665 Apr 12 16:42 custom_grep.c
-rwxrwxr-x 1 16832 Apr 12 18:11 custom_ls
-rw-rw-r-- 1  5967 Apr 12 18:11 custom_ls.c
-rwxrwxr-x 1 17208 Apr 12 16:52 custom_shell
-rw-rw-r-- 1  9391 Apr 12 16:42 custom_shell.c
-rw-rw-r-- 1 256356 Apr 12 16:42 sahil.pdf

Files: 13 | Directories: 0
custom_shell:~/G25_OS-Project/G25_Project2_1$
```

Screenshot 3 – Hidden Files

Command: **custom_ls -a**

```
custom_shell:~/G25_OS-Project/G25_Project2_1$ custom_ls -a
.
..
.gitkeep
Adyant.pdf
Makefile
Nayan.pdf
README.md
custom_cat
custom_cat.c
custom_grep
custom_grep.c
custom_ls
custom_ls.c
custom_shell
custom_shell.c
sahil.pdf

Files: 14 | Directories: 2
custom_shell:~/G25_OS-Project/G25_Project2_1$
```

Screenshot 4 – Combined Flags

Command: **custom_ls -lah**

```
custom_shell:~/G25_OS-Project/G25_Project2_1$ custom_ls -lah
drwxrwxr-x 2 4.0K Apr 12 18:11 .
drwxrwxr-x 5 4.0K Apr 12 16:42 ..
-rw-rw-r-- 1 1B Apr 12 16:42 .gitkeep
-rw-rw-r-- 1 3.9M Apr 12 16:42 Adyant.pdf
-rw-rw-r-- 1 2.3K Apr 12 16:42 Makefile
-rw-rw-r-- 1 531.2K Apr 12 16:42 Nayan.pdf
-rw-rw-r-- 1 10.2K Apr 12 16:42 README.md
-rwxrwxr-x 1 16.1K Apr 12 16:56 custom_cat
-rw-rw-r-- 1 3.3K Apr 12 16:42 custom_cat.c
-rwxrwxr-x 1 16.1K Apr 12 16:56 custom_grep
-rw-rw-r-- 1 1.6K Apr 12 16:42 custom_grep.c
-rwxrwxr-x 1 16.4K Apr 12 18:11 custom_ls
-rw-rw-r-- 1 5.8K Apr 12 18:11 custom_ls.c
-rwxrwxr-x 1 16.8K Apr 12 16:52 custom_shell
-rw-rw-r-- 1 9.2K Apr 12 16:42 custom_shell.c
-rw-rw-r-- 1 250.3K Apr 12 16:42 sahil.pdf

Files: 14 | Directories: 2
```

Screenshot 5 – Execution Inside Shell

Command: **custom_ls -lh**

```
custom_shell:~/G25_OS-Project/G25_Project2_1$ custom_ls -lh
-rw-rw-r-- 1 3.9M Apr 12 16:42 Adyant.pdf
-rw-rw-r-- 1 2.3K Apr 12 16:42 Makefile
-rw-rw-r-- 1 531.2K Apr 12 16:42 Nayan.pdf
-rw-rw-r-- 1 10.2K Apr 12 16:42 README.md
-rwxrwxr-x 1 16.1K Apr 12 16:56 custom_cat
-rw-rw-r-- 1 3.3K Apr 12 16:42 custom_cat.c
-rwxrwxr-x 1 16.1K Apr 12 16:56 custom_grep
-rw-rw-r-- 1 1.6K Apr 12 16:42 custom_grep.c
-rwxrwxr-x 1 16.4K Apr 12 18:11 custom_ls
-rw-rw-r-- 1 5.8K Apr 12 18:11 custom_ls.c
-rwxrwxr-x 1 16.8K Apr 12 16:52 custom_shell
-rw-rw-r-- 1 9.2K Apr 12 16:42 custom_shell.c
-rw-rw-r-- 1 250.3K Apr 12 16:42 sahil.pdf

Files: 13 | Directories: 0
custom_shell:~/G25_OS-Project/G25_Project2_1$
```

Limitations

- Does not support recursive listing (-R)
- Does not display owner/group names
- Does not support advanced formatting like columns

Future Improvements

- Add recursive directory listing (-R)
- Add reverse sorting (-r)
- Display owner and group names
- Add column-wise output like real ls
- Add file icons

Conclusion

The custom_ls utility successfully demonstrates the internal working of the UNIX ls command. It provides hands-on experience with directory traversal, file metadata handling, and command-line argument parsing.

The addition of advanced features such as sorting, color-coded output, and human-readable file sizes enhances usability and makes the implementation closer to real UNIX utilities.

This project significantly improved my understanding of system programming concepts and strengthened my skills in C programming and modular design.